

CURSO DE MLOPS

Tema 7 — CI/CD Básico Aplicado a ML

Caso Práctico: Trigger Automático al hacer Push a la Rama Main

Adaptado para GitHub Codespaces

Nivel	Base	Entorno	Duración
Intermedio	Tema 6 — Picos	GitHub Codespace	90 – 120 min

■ Adaptaciones para GitHub Codespaces

Aspecto	Original	GitHub Codespaces
Ejecución	Local/Colab	Terminal integrada del Codespace (Ctrl+`)
GitHub Actions	Se activa con push al repo	Idéntico — el Codespace tiene Git y push configurado
Docker	Docker Desktop local	Docker disponible en el Codespace — sin instalación
Make	make instalado localmente	make disponible en el Codespace
Secrets (DockerHub)	Variables locales	GitHub Secrets en Settings del repositorio
Rutas	Absolutas a tu máquina	Relativas al workspace /workspaces/<repo>
Badge CI/CD	README local	Se ve en GitHub automáticamente tras el primer push

- GitHub Actions corre en runners ubuntu-latest de GitHub (NO en el Codespace).
El Codespace se usa para: editar código, ejecutar make localmente y hacer git push.
- Cada git push a main dispara el workflow automáticamente en los servidores de GitHub.
- Prerrequisito: el repositorio debe estar en GitHub (no solo en el Codespace local).

¿Qué estamos haciendo y por qué?

En los Temas 1-6 construimos un pipeline ML completo: datos, modelos, tuning, tracking con MLflow y contenerización con Docker. El problema pendiente es la automatización: cada vez que un data scientist hace un

cambio al código, alguien tiene que recordar ejecutar los tests, re-entrenar el modelo, reconstruir la imagen Docker y validar que todo sigue funcionando.

En este tema implementamos CI/CD para ML: un workflow de GitHub Actions que se dispara automáticamente con cada git push a main y ejecuta en orden: linting (flake8), tests unitarios (pytest), entrenamiento del modelo, validación de métricas y build de la imagen Docker.

◆ Continuidad con el Tema 6

Usamos el proyecto de clasificación de picos de intensidad. Solo agregamos los archivos de CI/CD: `.github/workflows/ml_pipeline.yml`, `src/generate_data.py`, `src/train_pipeline.py`, `src/validate_model.py`, `tests/`, `Makefile` y `setup.cfg`.

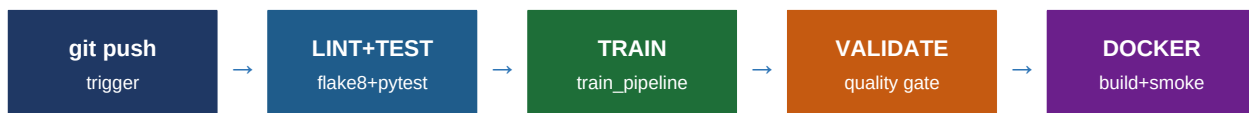
Al terminar lograrás:

- Entender qué es CI/CD y cómo aplica específicamente a proyectos ML
- Crear un workflow de GitHub Actions que se activa con push a main
- Configurar jobs de linting con flake8 y tests con pytest
- Automatizar el re-entrenamiento del modelo y validación de métricas
- Construir la imagen Docker automáticamente desde el repositorio
- Ejecutar todo el pipeline localmente con un Makefile antes de hacer push

Conceptos Clave — CI/CD en ML

Concepto	Definición general	En proyectos ML
CI — Continuous Integration	Integrar cambios frecuentemente con tests automáticos	Cada push dispara: linting + tests del pipeline + validación de métricas
CD — Continuous Delivery	Desplegar automáticamente si CI pasa	Build de imagen Docker + push a registry si el modelo supera el umbral
Trigger	Evento que inicia el pipeline	push a main, pull_request, schedule (cron), dispatch manual
Job	Unidad de trabajo dentro del workflow	lint-and-test, train-model, build-docker (en secuencia con needs:)
Runner	Máquina virtual donde corre el job	ubuntu-latest gratuito en GitHub Actions
Quality Gate	Umbral mínimo de calidad	validate_model.py: sys.exit(1) si Recall < 0.70

Pipeline CI/CD del Proyecto



⚠ Si falla cualquier etapa — el pipeline se detiene.
 Si pytest falla → no se re-entrena. Si métricas < umbral → no se construye Docker.
 Solo código validado y modelos que cumplen el estándar llegan a producción.

Estructura de archivos nuevos

Archivos nuevos en el Tema 7 — marcados con ←

```

picos-intensidad-mlops/
├── .github/
│   └── workflows/
│       └── ml_pipeline.yml ← NUEVO: workflow CI/CD completo
├── src/
│   ├── generate_data.py ← NUEVO: genera dataset sintético
│   ├── train_pipeline.py ← NUEVO: entrena con GridSearchCV + SMOTE
│   └── validate_model.py ← NUEVO: quality gate de métricas
├── tests/
│   ├── __init__.py
│   ├── test_data.py ← NUEVO: 7 tests del generador de datos
│   ├── test_pipeline.py ← NUEVO: 6 tests del entrenamiento
│   └── test_model.py ← NUEVO: 6 tests del modelo serializado
│       generado por train_pipeline.py (en .gitignore)
├── artifacts/
│   ├── modelo.pkl
│   └── metrics.json
├── Dockerfile ← del Tema 5 (sin cambios)
├── Makefile ← NUEVO: ejecutar todo localmente
├── requirements.txt ← actualizado: + pytest + flake8
├── setup.cfg ← NUEVO: config de flake8 y pytest
└── .gitignore ← actualizado
  
```

PASO

1

requirements.txt y setup.cfg — Configuración de herramientas

Archivos de configuración

requirements.txt

```

numpy>=1.23
pandas>=1.5
  
```

```

scikit-learn>=1.2
imbalanced-learn>=0.10
matplotlib>=3.6
mlflow>=2.0
scipy>=1.9

# CI/CD – testing y linting
pytest>=7.0
pytest-cov>=4.0
flake8>=6.0

```

setup.cfg

```

[flake8]
max-line-length = 100
exclude = .git,__pycache__,artifacts,mlruns,.github
ignore = E203, W503, D100, D104
per-file-ignores =
    tests/*: D

[tool:pytest]
testpaths = tests
addopts = -v --tb=short
python_files = test_*.py
python_classes = Test*
python_functions = test_*

```

Terminal del Codespace – instalar

```

cd /workspaces/tu-repo
pip install -r requirements.txt

# Verificar que flake8 y pytest están disponibles
flake8 --version
pytest --version

```

PASO

2

src/generate_data.py — Generador del dataset

[src/generate_data.py](#)

Genera el dataset sintético de picos de intensidad y lo guarda como CSV. El pipeline CI/CD lo llama en el Job 2 antes de entrenar:

```
src/generate_data.py
```

```

"""generate_data.py – Genera dataset sintético de picos de intensidad."""
import numpy as np
import pandas as pd
from pathlib import Path

DATA_PATH = Path('data/Clasificacion_picos_intensidad.csv')
N = 1500
RANDOM_STATE = 42

def generate(n: int = N, random_state: int = RANDOM_STATE) -> pd.DataFrame:
    """Genera el dataset sintético con la misma estructura que el original."""
    rng = np.random.default_rng(random_state)
    n0, n1 = int(n * 0.92), int(n * 0.08)

    df0 = pd.DataFrame({
        'Presion': rng.normal(180, 20, n0), 'Tonelaje': rng.normal(350, 40, n0),
        'Velocidad': rng.normal(1800, 150, n0), '%Solidos': rng.normal(72, 5, n0),
        'Potencia': rng.normal(800, 80, n0), 'F80': rng.normal(120, 15, n0),
        'Brazo': rng.normal(55, 8, n0), 'picos_intens': 0})
    df1 = pd.DataFrame({
        'Presion': rng.normal(220, 25, n1), 'Tonelaje': rng.normal(420, 50, n1),
        'Velocidad': rng.normal(2200, 200, n1), '%Solidos': rng.normal(78, 6, n1),
        'Potencia': rng.normal(950, 100, n1), 'F80': rng.normal(145, 18, n1),
        'Brazo': rng.normal(62, 10, n1), 'picos_intens': 1})
    return pd.concat([df0, df1]).sample(frac=1, random_state=random_state)

if __name__ == '__main__':
    DATA_PATH.parent.mkdir(exist_ok=True)
    df = generate()
    df.to_csv(DATA_PATH, index=False)
    print(f'Dataset generado: {df.shape} | tasa clase 1: {df.picos_intens.mean():.2%}')

```

PASO

3

src/train_pipeline.py — Pipeline de entrenamiento

src/train_pipeline.py

src/train_pipeline.py

```

"""train_pipeline.py – Entrena el modelo y guarda artefactos."""
import json, pickle, logging
from pathlib import Path
import pandas as pd
from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import f1_score, recall_score, accuracy_score
from imblearn.over_sampling import SMOTE

```

```

from generate_data import generate, DATA_PATH

ARTIFACTS = Path('artifacts')
MODEL_PATH = ARTIFACTS / 'modelo.pkl'
METRICS_PATH = ARTIFACTS / 'metrics.json'
FEATURES = ['Presion', 'Tonelaje', 'Velocidad', '%Solidos', 'Potencia', 'F80', 'Brazo']
TARGET = 'picos_intens'
RANDOM_STATE = 123
TEST_SIZE = 0.30
RECALL_MIN = 0.70

log = logging.getLogger(__name__)

def load_data() -> pd.DataFrame:
    """Carga o genera el dataset."""
    if not DATA_PATH.exists():
        generate()
    return pd.read_csv(DATA_PATH)

def train(df: pd.DataFrame) -> dict:
    """Entrena con GridSearchCV + SMOTE y retorna métricas."""
    X, y = df[FEATURES], df[TARGET]
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE, stratify=y)
    X_tr_s, y_tr_s = SMOTE(k_neighbors=5, random_state=RANDOM_STATE).fit_resample(
        X_train, y_train)
    param_grid = {
        'max_depth': [5, 8, None],
        'min_samples_split': [2, 5, 10],
        'criterion': ['gini', 'entropy'],
    }
    gs = GridSearchCV(DecisionTreeClassifier(random_state=RANDOM_STATE),
        param_grid, cv=StratifiedKFold(5), scoring='f1', n_jobs=-1)
    gs.fit(X_tr_s, y_tr_s)

    y_pred = gs.best_estimator_.predict(X_test)
    metricas = {
        'f1': round(f1_score(y_test, y_pred), 4),
        'recall': round(recall_score(y_test, y_pred), 4),
        'accuracy': round(accuracy_score(y_test, y_pred), 4),
        'params': gs.best_params_,
        'recall_minimo': RECALL_MIN,
    }
    ARTIFACTS.mkdir(exist_ok=True)
    with open(MODEL_PATH, 'wb') as f: pickle.dump(gs.best_estimator_, f)
    with open(METRICS_PATH, 'w') as f: json.dump(metricas, f, indent=2)
    return metricas

```

```
if __name__ == '__main__':
    metricas = train(load_data())
    print(json.dumps(metricas, indent=2))
```

PASO

4

src/validate_model.py — Quality gate de métricas

src/validate_model.py

Si Recall < 0.70, llama sys.exit(1). GitHub Actions interpreta exit code != 0 como fallo y detiene el pipeline — el Job de Docker no arranca:

src/validate_model.py

```
"""validate_model.py – Valida que las métricas superen el umbral mínimo.
Si falla, el pipeline CI/CD se detiene y no se construye la imagen Docker.
"""

import json, sys
from pathlib import Path

METRICS_PATH = Path('artifacts/metrics.json')

def validate(metrics_path: Path = METRICS_PATH) -> None:
    """Valida métricas. Lanza SystemExit(1) si falla el quality gate."""
    if not metrics_path.exists():
        print(f'ERROR: {metrics_path} no existe.')
        sys.exit(1)

    with open(metrics_path) as f:
        m = json.load(f)
        umbral = m.get('recall_minimo', 0.70)
        recall = m.get('recall', 0.0)

    print('='*50)
    print(' QUALITY GATE – VALIDACIÓN DE MÉTRICAS')
    print('='*50)
    print(f' Recall   : {recall:.4f} (umbral: >= {umbral})')
    print(f' F1-Score  : {m["f1"]:.4f}')
    print(f' Accuracy  : {m.get("accuracy", 0):.4f}')

    if recall < umbral:
        print(f'\n FALLO: Recall {recall:.4f} < umbral {umbral}')
        sys.exit(1)          # GitHub Actions detecta exit code 1 → fallo

    print(f'\n APROBADO: Recall {recall:.4f} >= {umbral}')
```

```
if __name__ == '__main__':
```

`validate()`

PASO

5

tests/ — Tests unitarios con pytest

19 tests en 3 archivos

tests/test_data.py — 7 tests del generador de datos**tests/test_data.py**

```

"""tests/test_data.py – Tests del generador de datos."""
import sys; sys.path.insert(0, 'src')
from generate_data import generate

FEATURES = ['Presion', 'Tonelaje', 'Velocidad', '%Solidos', 'Potencia', 'F80', 'Brazo']

def test_generate_returns_dataframe():
    assert isinstance(generate(n=200), __import__('pandas').DataFrame)

def test_generate_has_correct_columns():
    df = generate(n=200)
    for col in FEATURES + ['picos_intens']:
        assert col in df.columns

def test_generate_target_is_binary():
    assert set(generate(n=300)['picos_intens'].unique()).issubset({0, 1})

def test_generate_class_imbalance():
    rate = generate(n=500)['picos_intens'].mean()
    assert 0.05 <= rate <= 0.20

def test_generate_no_nulls():
    assert generate(n=300).isnull().sum().sum() == 0

def test_generate_n_rows():
    assert len(generate(n=400)) == 400

def test_generate_reproducible():
    df1, df2 = generate(n=200, random_state=99), generate(n=200, random_state=99)
    assert df1.reset_index(drop=True).equals(df2.reset_index(drop=True))

```

tests/test_pipeline.py — 6 tests del entrenamiento**tests/test_pipeline.py**


```

"""tests/test_pipeline.py — Tests del pipeline de entrenamiento."""
import json, sys; sys.path.insert(0, 'src')
from generate_data import generate
from train_pipeline import train

def test_train_returns_metrics(tmp_path, monkeypatch):
    monkeypatch.chdir(tmp_path)
    m = train(generate(n=300))
    assert 'f1' in m and 'recall' in m and 'accuracy' in m

def test_train_f1_positive(tmp_path, monkeypatch):
    monkeypatch.chdir(tmp_path)
    assert train(generate(n=300))['f1'] > 0.0

def test_train_metrics_in_range(tmp_path, monkeypatch):
    monkeypatch.chdir(tmp_path)
    m = train(generate(n=300))
    for key in ('f1', 'recall', 'accuracy'):
        assert 0.0 <= m[key] <= 1.0

def test_train_saves_model(tmp_path, monkeypatch):
    monkeypatch.chdir(tmp_path)
    train(generate(n=300))
    assert (tmp_path / 'artifacts' / 'modelo.pkl').exists()

def test_train_saves_metrics(tmp_path, monkeypatch):
    monkeypatch.chdir(tmp_path)
    train(generate(n=300))
    with open(tmp_path / 'artifacts' / 'metrics.json') as f:
        m = json.load(f)
    assert 'f1' in m and 'recall' in m

def test_train_saves_best_params(tmp_path, monkeypatch):
    monkeypatch.chdir(tmp_path)
    train(generate(n=300))
    with open(tmp_path / 'artifacts' / 'metrics.json') as f:
        assert isinstance(json.load(f)['params'], dict)

```

tests/test_model.py — 6 tests del modelo serializado

tests/test_model.py

```

"""tests/test_model.py — Tests del modelo serializado."""
import os, pickle, sys
import numpy as np
import pytest
sys.path.insert(0, 'src')

```

```

from generate_data import generate
from train_pipeline import train, FEATURES

@pytest.fixture(scope='module')
def modelo_entrenado(tmp_path_factory):
    tmp = tmp_path_factory.mktemp('workdir')
    os.chdir(tmp)
    train(generate(n=400))
    with open(tmp / 'artifacts' / 'modelo.pkl', 'rb') as f:
        return pickle.load(f)

def test_model_has_predict(modelo_entrenado):
    assert hasattr(modelo_entrenado, 'predict')

def test_model_has_predict_proba(modelo_entrenado):
    assert hasattr(modelo_entrenado, 'predict_proba')

def test_model_predicts_binary(modelo_entrenado):
    X = generate(n=50)[FEATURES].values
    assert set(modelo_entrenado.predict(X)).issubset({0, 1})

def test_model_predict_proba_shape(modelo_entrenado):
    X = generate(n=10)[FEATURES].values
    proba = modelo_entrenado.predict_proba(X)
    assert proba.shape == (10, 2) and (proba >= 0).all() and (proba <= 1).all()

def test_model_predict_proba_sums_to_one(modelo_entrenado):
    X = generate(n=20)[FEATURES].values
    np.testing.assert_allclose(modelo_entrenado.predict_proba(X).sum(axis=1), 1.0,
atol=1e-6)

def test_model_is_decision_tree(modelo_entrenado):
    from sklearn.tree import DecisionTreeClassifier
    assert isinstance(modelo_entrenado, DecisionTreeClassifier)

```

PASO

6

.github/workflows/ml_pipeline.yml — El corazón del CI/CD

GitHub Actions YAML — el workflow completo

Este es el archivo más importante del tema. GitHub lo ejecuta automáticamente cada vez que alguien hace push a la rama main. Define 3 jobs encadenados con needs: — si uno falla, los siguientes no arrancan:

```
.github/workflows/ml_pipeline.yml
```

```

# CI/CD Pipeline — MLOps Picos de Intensidad
name: ML Pipeline CI/CD

```

```

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
  workflow_dispatch: # ejecutar manualmente desde la UI

env:
  PYTHON_VERSION: '3.10'
  IMAGE_NAME: picos-intensidad-api

jobs:
  # — JOB 1: Lint y Tests —————
  lint-and-test:
    name: Lint + Tests
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${ env.PYTHON_VERSION }
          cache: 'pip'
      - name: Instalar dependencias
        run: pip install -r requirements.txt
      - name: Linting con flake8
        run: flake8 src/ tests/ --config=setup.cfg
      - name: Tests con pytest
        run: pytest tests/ -v --tb=short --cov=src --cov-report=term-missing
      - name: Subir reporte de cobertura
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: coverage-report
          path: .coverage

  # — JOB 2: Entrenar el modelo —————
  train-model:
    name: Entrenar Modelo
    runs-on: ubuntu-latest
    needs: lint-and-test # solo si el Job 1 pasó
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${ env.PYTHON_VERSION }
          cache: 'pip'
      - run: pip install -r requirements.txt

```

```

- name: Generar dataset
  run: python src/generate_data.py
- name: Entrenar pipeline
  run: python src/train_pipeline.py
- name: Validar métricas (quality gate)
  run: python src/validate_model.py
- name: Guardar artefactos
  uses: actions/upload-artifact@v4
  with:
    name: model-artifacts
    path: artifacts/

```

— JOB 3: Build Docker

```

build-docker:
  name: Build Docker Image
  runs-on: ubuntu-latest
  needs: train-model          # solo si el Job 2 pasó
  steps:
    - uses: actions/checkout@v4
    - name: Descargar artefactos del modelo
      uses: actions/download-artifact@v4
      with:
        name: model-artifacts
        path: artifacts/
    - name: Build de la imagen Docker
      run: |
        docker build -t ${ env.IMAGE_NAME }}:${ env.github.sha }} .
        docker build -t ${ env.IMAGE_NAME }}:latest .
    - name: Smoke test de la imagen
      run: |
        docker run --rm ${ env.IMAGE_NAME }}:latest python -c \
          'import pickle; m=pickle.load(open("artifacts/modelo.pkl","rb")); \
            print("Modelo OK:", type(m).__name__)'

```

PASO

7

Makefile — Ejecutar el pipeline localmente

Antes de hacer push — simula GitHub Actions

El Makefile permite ejecutar localmente los mismos pasos que GitHub Actions antes de hacer push. Así se detectan errores antes de que lleguen al repositorio:

Makefile

```

# Makefile — Pipeline CI/CD Local
.PHONY: all lint test train validate docker clean help

```

```

all: lint test train validate docker
    @echo '✓ Pipeline local completado. Listo para git push.'

lint:
    @echo '=== Linting con flake8 ==='
    flake8 src/ tests/ --config=setup.cfg

test:
    @echo '=== Tests con pytest ==='
    pytest tests/ -v --tb=short --cov=src --cov-report=term-missing

train:
    python src/generate_data.py
    python src/train_pipeline.py

validate:
    python src/validate_model.py

docker:
    docker build -t picos-intensidad-api:local .
    docker run --rm picos-intensidad-api:local python -c \
        "import pickle; m=pickle.load(open('artifacts/modelo.pkl','rb'));
print('OK:', type(m).__name__)"

clean:
    rm -rf artifacts/ data/ mlruns/ __pycache__ .coverage htmlcov/
    find . -name '*.pyc' -delete

```

Terminal del Codespace – ejecutar pipeline completo localmente

```

cd /workspaces/tu-repo

make all          # lint + test + train + validate + docker

# 0 por etapas:
make lint         # solo linting con flake8
make test         # solo tests con pytest
make train        # generar datos + entrenar modelo
make validate     # quality gate de métricas
make docker       # build y smoke test de la imagen
make clean        # limpiar artefactos y cachés

```

PASO

8

Activar el pipeline — git push desde el Codespace

[GitHub.com](#) → [tu repositorio](#) → [pestaña Actions](#)

```
# Terminal del Codespace – activar el pipeline CI/CD

cd /workspaces/tu-repo

# 1. Ejecutar el pipeline localmente primero (detecta errores antes del push)
make all

# 2. Subir a GitHub – esto activa el workflow automáticamente
git add .
git commit -m "feat: agregar CI/CD pipeline con GitHub Actions"
git push origin main

# 3. GitHub detecta el push y activa el workflow en sus servidores
#   Ve a: github.com/tu-usuario/tu-repo → pestaña Actions

# 4. Verás los 3 jobs ejecutándose en secuencia:
#   ✓ Lint + Tests           (flake8 + pytest)
#   ✓ Entrenar Modelo       (generate + train + validate)
#   ✓ Build Docker          (build + smoke test)
```

- Badge de CI/CD en el README.md — agregar esta línea:
![ML Pipeline](https://github.com/TU_USUARIO/TU_REPO/actions/workflows/ml_pipeline.yml/badge.svg)
El badge se actualiza automáticamente con el estado del último run.
- workflow_dispatch: permite ejecutar el pipeline manualmente desde la UI de GitHub Actions sin necesidad de hacer un push.

Referencia rápida de archivos y su rol en el pipeline

Archivo	Tipo	Rol en el pipeline CI/CD
ml_pipeline.yml	GitHub Actions	Orquesta los 3 jobs: lint-test → train → docker
generate_data.py	Python script	Genera el dataset sintético (Job 2: train)
train_pipeline.py	Python script	Entrena con GridSearchCV + SMOTE, guarda .pkl y .json
validate_model.py	Python script	Quality gate: sys.exit(1) si Recall < 0.70 (Job 2)
test_data.py	pytest (7 tests)	Valida estructura y calidad del dataset (Job 1)
test_pipeline.py	pytest (6 tests)	Valida que el entrenamiento produce artefactos (Job 1)
test_model.py	pytest (6 tests)	Valida que el modelo serializado predice correctamente (Job 1)
Makefile	Make	Ejecutar todo localmente antes del push

setup.cfg	Config	Reglas de flake8 y rutas de pytest
Dockerfile	Docker	Imagen que empaqueta el modelo (Job 3)

¿Qué aprendiste en este tema?

1. Qué es CI/CD en ML

Cada push a main activa automáticamente: linting, tests, re-entrenamiento, validación de métricas y build de Docker. Sin intervención manual. Solo código validado y modelos que cumplen el estándar llegan a producción.

2. GitHub Actions para ML

El archivo `ml_pipeline.yml` define 3 jobs encadenados con `needs`:. Si `lint-and-test` falla, `train-model` no arranca. Si `train` falla, no se construye Docker. El `workflow_dispatch` permite ejecución manual.

3. Linting y tests como guardianes de calidad

`flake8` valida el estilo del código con las reglas de `setup.cfg`. 19 tests con `pytest` validan el generador de datos, el pipeline de entrenamiento y el modelo serializado. `pytest-cov` mide la cobertura.

4. Quality gate de métricas

`validate_model.py` llama `sys.exit(1)` si `Recall < 0.70`. GitHub Actions interpreta `exit code != 0` como fallo y detiene el pipeline. El umbral se define en `RECALL_MIN` dentro del script y se guarda en `metrics.json`.

5. Build Docker automático en CI

El job `build-docker` descarga los artefactos del job anterior con `download-artifact`, construye la imagen y hace un smoke test para verificar que el modelo cargó correctamente dentro del contenedor.

◆ Próximos pasos — Monitoreo en Producción

Con el CI/CD en lugar, el siguiente paso es el monitoreo: detectar data drift, alertas cuando el modelo empeora en producción y re-entrenamiento automático programado con `scheduled trigger (cron)` en GitHub Actions.